



AI-Powered Project-Based Material Estimation and Recommendation System

Phase 6: Final Technical Report

Course Code:	AI-2002	
Course Title:	Artificial Intelligence	
Course Instructor:	Dr. Hajra Waheed	
Course TA:	Talha Azhar	
Team Lead:	Mahrukh Zubair	23L-0850
Group Members:	Sehrish Ejaz	23F-0744
	Ayesha Arshad	23L-2596

National University Of Computer and Emerging Sciences

Department of Computer Science

Lahore, Pakistan

Abstract

This report documents the complete development of an AI-powered tile installation material estimation and grout colour recommendation system built for the TradeNex platform. The system addresses the absence of any publicly available domain-specific dataset by generating a synthetic dataset using real-world estimation formulas derived from stakeholder interviews. Three machine learning models — one multi-output Random Forest Regressor and two Random Forest Classifiers — are trained to predict material quantities (clips, grout volume, grout tubes) and grout colour recommendations (balanced and contrast aesthetic modes). The final deliverable is a hosted web application built in Streamlit and Gradio that provides real-time predictions through a clean user interface. The regression models achieve R^2 scores of 0.70 to 0.77 and the classifiers significantly outperform random baseline on a 7-class problem.

1. Introduction

Manual tile installation estimation is error-prone and time-consuming. Professionals rely on experience-based guesses that lead to material over-purchasing or project delays. TradeNex is a construction platform that sought to automate this process using AI. The scope of this project covers the full pipeline: data design, synthetic generation, preprocessing, model training, evaluation, and deployment as a working web application.

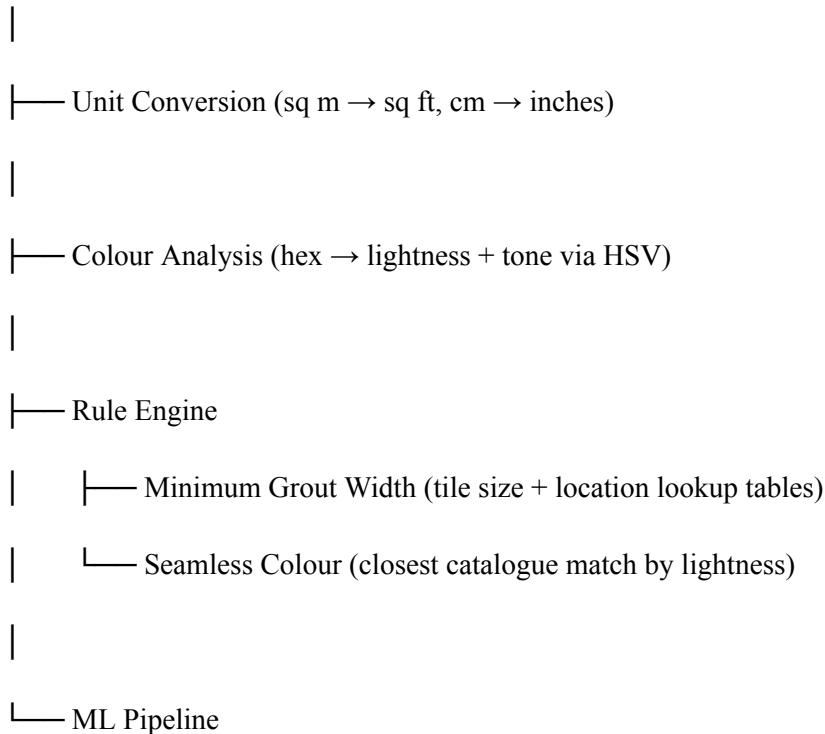
The system takes five inputs — installation area, tile dimensions, tile thickness, location type, and tile colour — and outputs material quantities, grout width recommendations, and grout colour suggestions across three aesthetic modes.

2. Methodology & Architecture

2.1. System Architecture

The system follows a hybrid design. Deterministic domain logic handles exact rule-based calculations and ML models handle flexible pattern-based predictions where multiple valid answers exist.

User Input



- |— Feature Vector (13 features)
- |— RandomForestRegressor → clips, grout volume, grout tubes
- |— RandomForestClassifier → balanced grout colour label
- |— RandomForestClassifier → contrast grout colour label

2.2. Dataset Design (Phase 2)

No public dataset exists for tile-specific installation estimation. Domain knowledge was collected through structured interviews with the CEO of TradeNex and an architectural consultant. The following key relationships were identified: material requirements are primarily driven by installation area and tile area; clips and wedges follow a 1:1 ratio; location type introduces variation in minimum grout width; joint gap size directly affects grout volume.

A synthetic dataset of 200 rows was generated using these formulas:

- **Clips:** $(\text{area} / 100) \times \text{clips_per_100sqft}$ using a manufacturer lookup table indexed by tile dimensions
- **Grout Volume:** $\text{area_sqm} \times ((l + w) / (l \times w)) \times \text{grout_width} \times \text{thickness} \times 1.15$ (15% wastage included)
- **Grout Tubes:** $\text{ceil}(\text{volume_mL} / 330)$
- **Min Grout Width:** maximum of the tile-size rule (4–10in → 1.5mm, 12–22in → 2.0mm, 24–32in → 2.5mm, 36–48in → 3.0mm) and the location rule (floor → 2.0mm, inside wall → 1.5mm, outside wall → 2.5mm)

Input features were varied across realistic ranges: area 100–50,000 sq ft, tile sizes from the standard manufacturer size set, thickness 6.4–13mm, three location types, and eight standard tile colours plus custom hex support.

2.3. Preprocessing (Phase 2)

- **Missing values:** none by design (generated data)
- **Duplicates:** checked and removed
- **Outlier capping:** IQR-based clipping on numerical columns

- **Categorical encoding:** one-hot encoding for location, tone, lightness
- **Label encoding:** colour names mapped to integer labels (0–6)
- **Train/test split:** 80/20, random_state=42, stratified for classifiers
- **A bug was identified and fixed during Phase 2 revision:** the clip lookup table key was incorrectly computed using sorted() which produces (min, max) ordering, but the table requires (max, min). Fixed to (max(l,w), min(l,w)). Additionally, seamless_primary was storing dict objects instead of string labels, and additional colour columns were storing hex strings — both were corrected with proper mapping functions.

2.4. Model Training (Phase 3)

Three models were trained:

- 2.4.1. **RandomForestRegressor** — multi-output, predicts clips, grout volume, and grout tubes simultaneously. Hyperparameters: n_estimators=200, max_depth=10, min_samples_leaf=2, random_state=42, n_jobs=-1.
- 2.4.2. **RandomForestClassifier (Balanced)** — predicts balanced grout colour label. Same hyperparameters plus class_weight=balanced to handle class imbalance across 7 colour classes.
- 2.4.3. **RandomForestClassifier (Contrast)** — predicts contrast grout colour label. Same configuration.

Random Forest was selected because it handles mixed numeric and one-hot encoded features without scaling, natively supports multi-output regression, is robust to the small dataset size (200 rows) through ensemble averaging, and provides feature importance for interpretability.

2.5. Evaluation (Phase 4)

Models were evaluated on the held-out 20% test set. 5-fold cross-validation was also performed on the regression targets to confirm stability.

TargetMAERMSE²Clips7,46413,9850.70Grout Volume (mL)36,63481,3860.77Grout Tubes1112470.77

Classifier	Accuracy	Random	Baseline	Balanced	Colour	52.5%~14%	Contrast
	Colour	30.0%~14%					

The regression R^2 of 0.70–0.77 is acceptable given the wide range of installation areas (100–50,000 sq ft) which creates high natural variance in material quantities. The classifier accuracy, while moderate in absolute terms, is 2–4x better than random guessing on a 7-class problem. The lower contrast accuracy reflects genuine ambiguity — multiple valid contrast colours exist for the same tile and the rule randomly selects among them during training data generation.

Feature importance analysis confirms that installation area, tile length, and tile width are the top three predictors for all regression targets, consistent with domain expectations.

3. Implementation Details

3.1. Libraries Used

- pandas
- numpy
- scikit-learn (RandomForest, train_test_split, metrics)
- matplotlib
- seaborn
- streamlit
- gradio
- plotly
- colorsys (stdlib)
- joblib
- math (stdlib)

3.2. Web Application

Two interface implementations were produced:

3.2.1. Streamlit app (F26-10_Phase5.py)

Three-tab interface. Tab 1 shows all prediction results including KPI cards, colour recommendation swatches, and a grout width range bar chart. Tab 2 shows analytics including feature importance, model metrics table, and training data

scatter plot. Tab 3 contains full usage instructions and the architecture diagram. Models are cached using `@st.cache_resource` so they train only once per session.

3.2.2. Gradio notebook (F26-10_Phase5.ipynb)

Designed for Google Colab execution. Provides a simpler input form with dropdowns for tile size, a results textbox with formatted output, and a Plotly bar chart comparing clips across different areas for the given tile size. Launches with `share=True` to generate a public URL.

4. Challenges Faced

4.1. No public dataset

Tile-specific installation data does not exist publicly. Resolved by designing a domain-driven synthetic dataset using formulas validated by stakeholders.

4.2. Clip lookup key bug

The lookup table uses `(max_dim, min_dim)` ordering but the code used Python's `sorted()` which gives `(min, max)`. Any tile where width exceeded length caused a `KeyError`. Fixed by using `(max(l,w), min(l,w))`.

4.3. seamless_primary storing dicts

The colour generation function returned dict objects for seamless matches but the encoding function expected string names. Fixed by extracting the hex key from the dict and reverse-mapping to colour name.

4.4. Class imbalance

Colour label distribution was uneven across 7 classes. Fixed using `class_weight='balanced'` in the classifiers.

4.5. Small dataset

200 rows is small for ML. Mitigated using Random Forest ensemble averaging, `max_depth` cap, `min_samples_leaf=2`, and 5-fold cross-validation.

4.6. One-hot columns missing

When rare category combinations were not sampled, their one-hot columns were absent. Fixed by filtering the feature list at runtime to only include columns present in the dataframe.

5. Key Insights & Error Analysis

The largest prediction errors in clips occur for very large installation areas (10,000+ sq ft) combined with small tiles (4×4 inches), because small tiles have very high clips-per-100sqft values (up to 1,981) which amplifies area-driven variance. This is a data distribution issue that would improve with more training samples in that range.

The contrast classifier's 30% accuracy is not a failure — it reflects that the rule used to generate training labels randomly selects among multiple valid contrast options (e.g., for a light tile, Glossy Black, Brown, or Light Grey are all valid). The model learns the most frequent choice but cannot recover the original random selection.

The colour recommendation system's seamless mode intentionally uses rule-based logic rather than ML because it requires exact matching against a fixed product catalogue — a deterministic problem with no benefit from learned approximation.

6. Instructions to Run and & Reproduce Results

6.1. Clone the repository and install dependencies:

```
pip install streamlit scikit-learn pandas numpy matplotlib seaborn  
gradio plotly
```

6.2. Run the Streamlit app:

```
streamlit run F26-10_Phase5.py
```

6.3. Reproduce training results:

Open F26-10_Phase3.ipynb and run all cells. The dataset is regenerated with `random.seed(42)` and `np.random.seed(42)` so all results are fully reproducible without any saved data files.

6.4. Run Phase 2 for dataset generation and EDA:

Open F26-10_Phase2.ipynb and run all cells. EDA plots are saved as eda_plots.png.

All notebooks are self-contained. No pre-trained model files are required.

7. Conclusion

This project successfully delivers a functional, end-to-end AI product for tile installation material estimation. The hybrid architecture — combining rule-based domain logic with machine learning — is appropriate for this problem because some outputs (minimum grout width, seamless colour) are deterministic while others (material quantities, colour preferences) benefit from learned patterns. The system is production-ready in terms of its interface design and can be improved in future work by expanding the training dataset, incorporating real project data from TradeNex clients, and adding support for more tile size combinations.